DBMS ASSIGNMENT

Task Name 3: Seller and Inventory Management System

---

1. Title

Seller and Inventory Management System

---

2. Introduction

A **Seller and Inventory Management System** is a database application used to manage sellers, products, and the available stock of products. The system helps an organization maintain accurate information about sellers and their products and monitor inventory levels.

The database stores seller details, product information, stock quantities, and product availability. Relationships are established between sellers, products, and inventory so that the organization can easily identify which seller supplies a particular product and how much stock is currently available.

The system can also generate inventory status reports that show whether products are **Available** or **Unavailable** based on their stock quantity.

---

3. Objectives

The main objectives of the Seller and Inventory Management System are:

1. To create and maintain a **Seller** table.
2. To create and maintain a **Product** table.
3. To create an **Inventory** table for tracking product stock.
4. To establish relationships between sellers, products, and inventory.
5. To maintain seller product information.
6. To track available and unavailable products.
7. To monitor current stock quantities.
8. To generate inventory status reports.
9. To reduce errors in manual inventory management.
10. To provide quick and accurate information about stock availability.

---

4. Database Design

The system consists of three main tables:

4.1 Seller

The Seller table stores information about sellers who provide products.

Attributes:

- Seller_ID – Unique identification number of the seller.
- Seller_Name – Name of the seller.
- Email – Email address of the seller.
- Phone – Contact number of the seller.
- Address – Address of the seller.

4.2 Product

The Product table stores information about products.

Attributes:

- Product_ID – Unique identification number of the product.
- Product_Name – Name of the product.
- Category – Category of the product.
- Price – Price of the product.
- Seller_ID – Identifies the seller supplying the product.

4.3 Inventory

The Inventory table stores stock information for each product.

Attributes:

- Inventory_ID – Unique identification number of the inventory record.
- Product_ID – Identifies the product.
- Quantity – Current available quantity.
- Reorder_Level – Minimum quantity at which stock should be reordered.
- Last_Updated – Date on which inventory information was last updated.

---

5. Relationship Between Tables

The relationships between the tables are as follows:

Seller → Product

One seller can supply many products.

Relationship: One-to-Many (1:N)

```
SELLER
   |
   | 1
   |
   |------< MANY
          PRODUCT
```

Product → Inventory

Each product has an inventory record that stores its current stock information.

Relationship: One-to-One (1:1) in this implementation.

```
PRODUCT
   |
   | 1
   |
   |------ 1
          INVENTORY
```

Overall Database Relationship

```
+----------------+
|     SELLER     |
+----------------+
| PK Seller_ID   |
| Seller_Name    |
| Email          |
| Phone          |
| Address        |
+----------------+
        |
        | 1
        |
        | M
+----------------+
|    PRODUCT     |
+----------------+
| PK Product_ID  |
| Product_Name   |
| Category       |
| Price          |
| FK Seller_ID   |
+----------------+
        |
        | 1
        |
        | 1
+----------------------+
|      INVENTORY       |
+----------------------+
| PK Inventory_ID     |
| FK Product_ID       |
| Quantity             |
```

```
| Reorder_Level         |
| Last_Updated          |
+-----------------------+
```

---

6. SQL Database Creation

The following SQL commands can be used to create the database and tables.

Step 1: Create Database

```
CREATE DATABASE SellerInventoryDB;

USE SellerInventoryDB;
```

---

7. Create Seller Table

```
CREATE TABLE Seller (
    Seller_ID INT PRIMARY KEY,
    Seller_Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15),
    Address VARCHAR(200)
);
```

Explanation

The `Seller_ID` is the primary key and uniquely identifies each seller. The `Seller_Name` field cannot be empty because it is defined as `NOT NULL`. The email address is defined as `UNIQUE` to avoid duplicate email addresses.

---

8. Create Product Table

```
CREATE TABLE Product (
    Product_ID INT PRIMARY KEY,
    Product_Name VARCHAR(100) NOT NULL,
    Category VARCHAR(50),
    Price DECIMAL(10,2) NOT NULL,
    Seller_ID INT,
    FOREIGN KEY (Seller_ID) REFERENCES Seller(Seller_ID)
);
```

Explanation

The `Product_ID` uniquely identifies each product. `Seller_ID` is a foreign key that connects the Product table with the Seller table.

This relationship allows the system to identify which seller supplies each product.

---

9. Create Inventory Table

```
CREATE TABLE Inventory (
    Inventory_ID INT PRIMARY KEY,
    Product_ID INT UNIQUE,
    Quantity INT NOT NULL,
    Reorder_Level INT NOT NULL,
    Last_Updated DATE,
    FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
);
```

Explanation

The `Inventory_ID` is the primary key. `Product_ID` is a foreign key that connects the Inventory table with the Product table.

The `UNIQUE` constraint on `Product_ID` ensures that one product has only one inventory record in this design.

---

10. Insert Seller Information

```
INSERT INTO Seller
(Seller_ID, Seller_Name, Email, Phone, Address)
VALUES
(101, 'ABC Electronics', 'abc@gmail.com', '9876543210', 'Chennai'),
(102, 'Global Traders', 'global@gmail.com', '9876543211', 'Bangalore'),
(103, 'Smart Solutions', 'smart@gmail.com', '9876543212', 'Hyderabad'),
(104, 'Tech World', 'techworld@gmail.com', '9876543213', 'Mumbai'),
(105, 'Digital Mart', 'digital@gmail.com', '9876543214', 'Delhi');
```

---

11. Insert Product Information

```
INSERT INTO Product
(Product_ID, Product_Name, Category, Price, Seller_ID)
VALUES
(201, 'Laptop', 'Electronics', 55000.00, 101),
(202, 'Wireless Mouse', 'Accessories', 800.00, 101),
(203, 'Keyboard', 'Accessories', 1200.00, 102),
(204, 'Smartphone', 'Electronics', 25000.00, 103),
(205, 'Headphones', 'Accessories', 2000.00, 104),
(206, 'Tablet', 'Electronics', 18000.00, 105);
```

---

12. Insert Inventory Information

```
INSERT INTO Inventory
(Inventory_ID, Product_ID, Quantity, Reorder_Level, Last_Updated)
VALUES
(301, 201, 15, 5, '2026-08-01'),
(302, 202, 50, 10, '2026-08-02'),
(303, 203, 0, 10, '2026-08-03'),
(304, 204, 8, 5, '2026-08-04'),
(305, 205, 0, 5, '2026-08-05'),
(306, 206, 20, 5, '2026-08-06');
```

---

13. Display Seller Details

```
SELECT * FROM Seller;
```

This query displays all seller information stored in the Seller table.

---

14. Display Product Details

```
SELECT * FROM Product;
```

This query displays all products available in the database.

---

15. Display Inventory Details

```
SELECT * FROM Inventory;
```

This query displays the current inventory information for all products.

---

16. Display Seller and Product Information

A JOIN operation can be used to display the seller name along with the products supplied by the seller.

```
SELECT
    S.Seller_ID,
    S.Seller_Name,
    P.Product_ID,
    P.Product_Name,
    P.Category,
```

```
    P.Price
FROM Seller S
JOIN Product P
ON S.Seller_ID = P.Seller_ID;
```

Purpose

This query helps identify which seller is supplying each product.

---

17. Display Product and Stock Information

```
SELECT
    P.Product_ID,
    P.Product_Name,
    P.Category,
    I.Quantity,
    I.Reorder_Level,
    I.Last_Updated
FROM Product P
JOIN Inventory I
ON P.Product_ID = I.Product_ID;
```

This query displays product information together with its current inventory quantity.

---

18. Track Available Products

A product is considered **Available** when its quantity is greater than zero.

```
SELECT
    P.Product_ID,
    P.Product_Name,
    I.Quantity,
    'Available' AS Status
FROM Product P
JOIN Inventory I
ON P.Product_ID = I.Product_ID
WHERE I.Quantity > 0;
```

Expected Result

Product ID	Product Name	Quantity	Status
201	Laptop	15	Available
202	Wireless Mouse	50	Available
204	Smartphone	8	Available
206	Tablet	20	Available

---

19. Track Unavailable Products

A product is considered **Unavailable** when its quantity is zero.

```
SELECT
    P.Product_ID,
    P.Product_Name,
    I.Quantity,
    'Unavailable' AS Status
FROM Product P
JOIN Inventory I
ON P.Product_ID = I.Product_ID
WHERE I.Quantity = 0;
```

Expected Result

Product ID	Product Name	Quantity	Status
203	Keyboard	0	Unavailable
205	Headphones	0	Unavailable

---

20. Generate Complete Inventory Status Report

The `CASE` statement can be used to generate an inventory status report.

```
SELECT
    P.Product_ID,
    P.Product_Name,
    S.Seller_Name,
    I.Quantity,
    I.Reorder_Level,
    CASE
        WHEN I.Quantity = 0 THEN 'Unavailable'
        WHEN I.Quantity <= I.Reorder_Level THEN 'Low Stock'
        ELSE 'Available'
    END AS Inventory_Status
FROM Product P
JOIN Seller S
ON P.Seller_ID = S.Seller_ID
JOIN Inventory I
ON P.Product_ID = I.Product_ID;
```

Sample Inventory Status Report

Product	Seller	Quantity	Reorder Level	Status
Laptop	ABC Electronics	15	5	Available
Wireless Mouse	ABC Electronics	50	10	Available
Keyboard	Global Traders	0	10	Unavailable

Product	Seller	Quantity	Reorder Level	Status
Smartphone	Smart Solutions	8	5	Available
Headphones	Tech World	0	5	Unavailable
Tablet	Digital Mart	20	5	Available

---

21. Find Products That Need Reordering

Products whose quantity is equal to or less than the reorder level should be reordered.

```
SELECT
    P.Product_ID,
    P.Product_Name,
    I.Quantity,
    I.Reorder_Level
FROM Product P
JOIN Inventory I
ON P.Product_ID = I.Product_ID
WHERE I.Quantity <= I.Reorder_Level;
```

This query helps the organization identify products that require additional stock.

---

22. Find Products Supplied by a Particular Seller

For example, to find products supplied by seller ID 101:

```
SELECT
    P.Product_ID,
    P.Product_Name,
    P.Category,
    P.Price
FROM Product P
WHERE P.Seller_ID = 101;
```

---

23. Count Products Supplied by Each Seller

```
SELECT
    S.Seller_ID,
    S.Seller_Name,
    COUNT(P.Product_ID) AS Total_Products
FROM Seller S
LEFT JOIN Product P
ON S.Seller_ID = P.Seller_ID
GROUP BY S.Seller_ID, S.Seller_Name;
```

This query provides the number of products supplied by each seller.

---

24. Calculate Total Inventory Value

The total inventory value can be calculated by multiplying product price by available quantity.

```
SELECT
    P.Product_ID,
    P.Product_Name,
    P.Price,
    I.Quantity,
    (P.Price * I.Quantity) AS Total_Inventory_Value
FROM Product P
JOIN Inventory I
ON P.Product_ID = I.Product_ID;
```

---

25. Find the Most Expensive Product

```
SELECT *
FROM Product
WHERE Price = (
    SELECT MAX(Price)
    FROM Product
);
```

This query identifies the product with the highest price.

---

26. Update Inventory Quantity

When new stock is received, the inventory quantity can be updated.

```
UPDATE Inventory
SET Quantity = 25,
    Last_Updated = '2026-08-11'
WHERE Product_ID = 201;
```

This updates the stock quantity of the Laptop to 25.

---

27. Reduce Inventory After a Sale

When a product is sold, its inventory quantity can be reduced.

```
UPDATE Inventory
SET Quantity = Quantity - 1,
    Last_Updated = '2026-08-11'
WHERE Product_ID = 201
AND Quantity > 0;
```

The condition `Quantity > 0` prevents the quantity from becoming negative.

---

28. Delete a Product

A product can be deleted using the following command:

```
DELETE FROM Product
WHERE Product_ID = 206;
```

Note: In a real system, the corresponding inventory record should be handled before deleting the product because of the foreign-key relationship.

---

29. Important SQL Concepts Used

The project demonstrates several important DBMS concepts:

Primary Key

A primary key uniquely identifies each record in a table.

Example:

```
Seller_ID INT PRIMARY KEY
```

Foreign Key

A foreign key establishes a relationship between two tables.

Example:

```
FOREIGN KEY (Seller_ID)
REFERENCES Seller(Seller_ID)
```

JOIN

JOIN is used to retrieve related information from multiple tables.

WHERE

The `WHERE` clause filters records based on a specific condition.

GROUP BY

`GROUP BY` is used to group records for aggregate calculations such as `COUNT()`.

CASE

`CASE` is used to classify inventory as Available, Low Stock, or Unavailable.

Aggregate Functions

Functions such as `COUNT()` and `MAX()` are used to perform calculations on data.

---

30. Advantages of the System

The Seller and Inventory Management System provides the following advantages:

1. Easy management of seller information.
2. Easy management of product information.
3. Accurate stock tracking.
4. Quick identification of unavailable products.
5. Identification of products requiring reordering.
6. Reduced manual errors.
7. Better relationship management between sellers and products.
8. Easy generation of inventory reports.
9. Improved inventory monitoring.
10. Faster access to important business information.

---

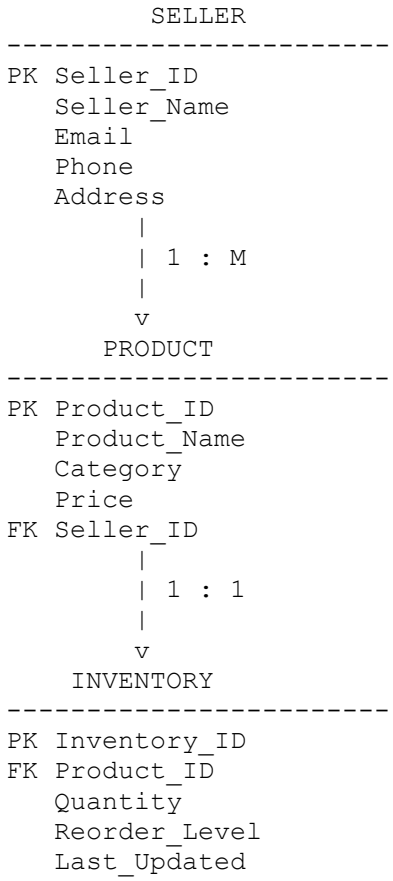
31. Conclusion

The **Seller and Inventory Management System** provides an organized way to manage sellers, products, and inventory using a relational database. The system establishes relationships between sellers and products and connects products with their corresponding inventory information.

Using SQL queries, the system can identify available and unavailable products, monitor low-stock products, calculate inventory value, and generate useful inventory status reports. Primary keys and foreign keys maintain data integrity and ensure proper relationships between tables.

Thus, the proposed database system provides a simple, efficient, and reliable solution for managing seller and inventory information.

32. Final Database Structure



Tables Used

Seller Table → Stores seller information.

Product Table → Stores product and seller information.

Inventory Table → Stores stock quantity and inventory status information.

Main Functions

Seller Management → Product Management → Inventory Management → Availability Tracking → Inventory Status Reporting